

07 — Install and use a conda environment on Windows

Question and scope

How do I install conda if necessary, create a separate Python environment, and use the correct interpreter in VS Code?

Anaconda/Miniconda supplies conda, the environment and package manager. A conda environment holds a Python interpreter and its packages. VS Code is the editor; choosing an interpreter connects it to an environment. This Windows walkthrough uses a new environment named `handbook-demo`; it does not change an existing research environment. Commands are instructions for you to run, not a record that an installation was performed.

Documentation was checked on 2026-09-20.

1. Check whether conda is already installed

Open **Anaconda Prompt** from the Windows Start menu and run:

```
conda --version
conda env list
```

If both work, skip reinstallation. An asterisk marks the active environment. A missing conda command in an ordinary terminal can mean that shell is not initialized, rather than that conda is absent.

Screenshot placeholder 07-1 — Anaconda Prompt showing conda --version and conda env list. Suggested file: `docs/images/07-conda-available.png`

If no conda installation exists, use the [official Windows installation guide](#)^[1] and a matching installer from [Anaconda's download page](#)^[2]. Miniconda provides a smaller starting installation; Anaconda Distribution includes more preinstalled packages. For a personal machine, use a per-user installation where appropriate. Finish the installer, open a fresh Anaconda Prompt, and repeat the two checks above. Keep environments outside your Git repository.

2. Create a project environment

The example chooses Python 3.11 for an isolated teaching environment. For a real project, use the version required by its dependencies or its existing environment file. First make sure `handbook-demo` is not already listed; if it exists, inspect it or choose a different name.

Run each command after the previous one succeeds, and review conda's proposed package changes when prompted:

```
conda create -n handbook-demo python=3.11 numpy matplotlib pip
conda activate handbook-demo
python --version
python -c "import sys; print(sys.executable)"
python -c "import numpy, matplotlib; print(numpy.__version__, matplotlib.__version__)"
```

Check that `sys.executable` points into the new environment. A successful import verifies that both packages are available to that interpreter. Do not rely only on the prompt prefix. See [conda environment management](#)^[3].

Screenshot placeholder 07-2 — Activated handbook-demo environment and interpreter/version verification.

Suggested file: docs/images/07-conda-active.png

`conda activate handbook-demo` selects an existing environment; it does not install anything. `conda deactivate` leaves it. If activation fails in PowerShell, run `conda init powershell` from Anaconda Prompt, then close and reopen PowerShell. This initializes that shell's startup configuration. If local policy prevents startup scripts, use Anaconda Prompt rather than changing system-wide security settings as a first response.

3. Select the interpreter in VS Code

Install Microsoft's Python extension if it is not already available. Open the project, press **Ctrl+Shift+P**, run **Python: Select Interpreter**, and select `handbook-demo`. If necessary, enter the interpreter path printed by `sys.executable`. Open a new terminal and repeat the interpreter/import checks. Existing terminals may still have the previous environment active. See [VS Code's environment guide](#) ^[4].

Screenshot placeholder 07-3 — VS Code interpreter picker with `handbook-demo` selected. Suggested file:

docs/images/07-vscode-interpreter.png

For a notebook, its kernel is a separate selection: choose the intended environment through **Select Kernel**. If it needs an IPython kernel, install `ipykernel` in that environment and select it again:

```
conda install -n handbook-demo ipykernel
```

4. Save a reproducible starting specification

Create a UTF-8 `environment.yml` in your project, for example:

```
name: handbook-demo
channels:
  - defaults
dependencies:
  - python=3.11
  - numpy
  - matplotlib
  - pip
```

To reproduce this specification in a fresh environment, use `conda env create -f environment.yml`. If the name already exists, use a new one, for example `conda env create -n handbook-demo-copy -f environment.yml`. This recipe intentionally leaves most package versions open; it is not an exact lock file. Record the resolved versions when comparing research results.

To export the current environment's explicitly requested packages in PowerShell:

```
conda env export -n handbook-demo --from-history | Set-Content -Encoding utf8 environment.yml
```

This overwrites that file; use a different filename if you want to retain the hand-written recipe. Inspect the export before sharing: remove any machine-specific `prefix: path`. The history export is useful for portability but does not capture every resolved dependency or every package installed with `pip`. Channel choice, operating system, and package availability also affect reconstruction.

If pip is needed, install conda packages first and then use `python -m pip install PACKAGE_NAME` inside the activated environment, replacing `PACKAGE_NAME` with the actual package. Record these additional dependencies; prefer rebuilding from a complete specification when changing a mixed conda/pip environment.

5. Troubleshooting and completion check

Symptom	First check
conda not recognized	Use Anaconda Prompt; check shell initialization
<code>ModuleNotFoundError</code>	Print <code>sys.executable</code> ; install in that environment
VS Code runs another Python	Select the interpreter, then open a new terminal
Notebook uses another environment	Check its kernel selection
Dependency solving fails	Check Python/package compatibility and project requirements

You are ready when the environment is listed, its interpreter imports the required packages, VS Code uses that interpreter, and the project has a reviewed environment specification.

6. Replace the screenshot placeholders

Save future screenshots under the three suggested names in `docs/images/`. Replace each visible block with the image line in its adjacent HTML comment, removing the comment markers. The bilingual guides share those image files; no nonexistent images are linked yet. Crop personal paths if sharing screenshots publicly.

Regenerate with `python tools/export_print.py docs/07-main-conda-environment.md`.

References

1. <https://docs.conda.io/projects/conda/en/latest/user-guide/install/windows.html>
2. <https://www.anaconda.com/download>
3. <https://docs.conda.io/projects/conda/en/stable/user-guide/tasks/manage-environments.html>
4. <https://code.visualstudio.com/docs/python/environments>